

TRUTHSEEKERS

Engine for Evidence-Grounded Knowledge

CONFIDENTIAL

June 2026 · veritas/docs/EXECUTIVE_SUMMARY.html

Truthseekers is a knowledge construction engine that produces structured, evidence-grounded encyclopedia articles — not chat summaries, not Wikipedia mirrors. It actively surfaces evidence across all truth categories (confirmed, contested, suppressed, speculative), exposes framing and structural distortion, and refuses to treat the absence of evidence as silence.

3

STRICT EPISTEMIC LAYERS

9

PIPELINE NODES

14

AGENT TOOLS

23

STORAGE CRUD OPS

The Problem

Every knowledge platform today has the same architecture: a chatbot with a search query. The result is a summary — fluent but flat, confident but shallow, unable to distinguish between consensus, controversy, and deliberate suppression. Wikipedia, the gold standard, is a political instrument with gatekept narratives. LLMs amplify the problem: they reproduce the most statistically common framing, which is the establishment framing.

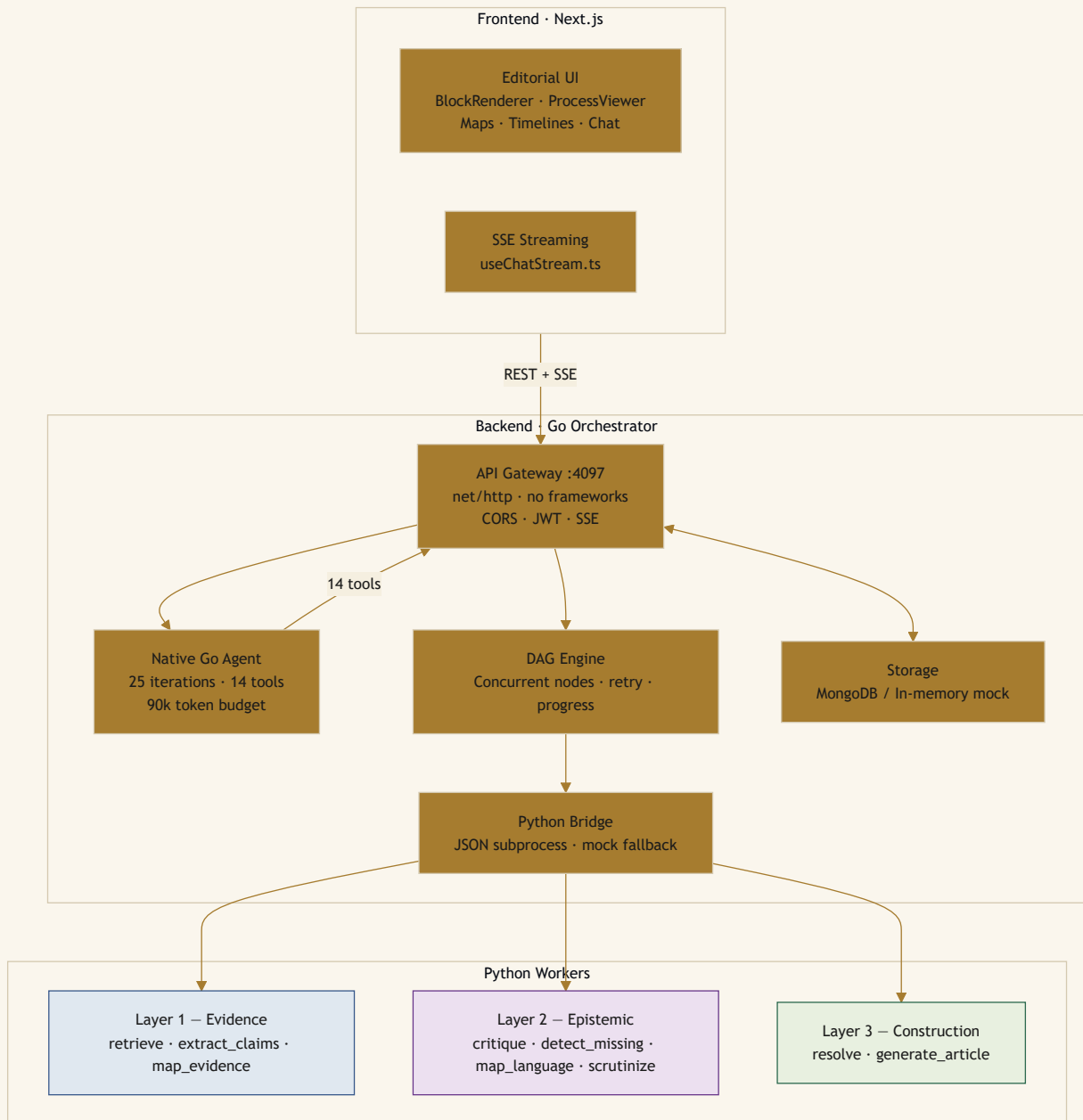
Conventional systems cannot:

- Distinguish evidence quality from narrative fluency
- Detect missing evidence as a meaningful signal
- Separate factual claims from framing/language choices
- Trace provenance from final article back to source passages
- Assign multi-dimensional confidence (not just "this is true")
- Surface suppressed or contested evidence alongside consensus

Truthseekers does all six, by design:

- Strict three-layer epistemic separation — evidence, analysis, construction never mix roles
- Gap detection as a first-class epistemic signal — "dogs that didn't bark"
- Dual-output language mapping — original framing preserved alongside precision upgrade
- Atomic claim graph with full provenance chains
- Six-axis confidence: evidence strength, corroboration, source diversity, recency, contradiction level, bias risk
- Four truth categories: confirmed, contested, suppressed, speculative — all retrieved

Architecture in Brief



Go

BACKEND

Zero-framework `net/http` API gateway on port 4097. Native agent loop, DAG engine, Python subprocess bridge, MongoDB with in-memory mock mode. Graceful shutdown, CORS, HMAC JWT auth.

`stdlib only`

Python

WORKERS

9 stateless epistemic nodes. Each is a standalone script, JSON-in/JSON-out over STDIN/STDOUT. Shared LLM client with 4-provider fallback chain. Mock responses when no API key is set. `subprocess bridge`

Next.js

Zero

FRONTEND

React 19, Framer Motion, TanStack Query, Tailwind CSS. Editorial design with antique gold & ink aesthetic. Structured block renderer, SSE streaming chat, 3D map viewer, interactive timeline, Mermaid.js diagrams. `live data`

INFRASTRUCTURE

Full local development with no external dependencies. Mock MongoDB, mock Python workers, mock web search. Meaningful demo from `go run ./cmd/server` alone. No Docker, no cloud account required. `zero-infra mode`

Technical Differentiation

STRICT EPISTEMIC LAYERS

Three layers with typed I/O contracts, enumerated permissions, and hard prohibitions enforceable at code level. Layer 1 detects evidence but cannot assign causality. Layer 2 interprets but must self-label as interpretive. Layer 3 synthesizes but cannot introduce new claims.

CLAIM GRAPH ARCHITECTURE

Claims are first-class objects with full provenance chains. Articles are generated views over an atomic claim graph. Every assertion traces to a source passage. Six-axis confidence vectors replace scalar "confidence scores."

GAP DETECTION

`detect_missing.py` identifies structural evidence gaps — "dogs that didn't bark." Absence is a meaningful epistemic signal, not silence. Gaps are labeled by type and verification status, with cause attribution only when external metadata exists.

LANGUAGE MAPPING

`map_language.py` detects euphemism, framing, and linguistic distortion. Outputs dual-channel: original language preserved alongside precision upgrade. "Collateral damage" maps to "civilian deaths." "Enhanced interrogation" maps to "torture." No information lost.

MULTI-PROVIDER LLM ROUTING

Four fallback providers in Python workers (Groq → DO Inference → NVIDIA NIM → OpenAI), two in Go agent (Groq + DO Inference). Deterministic `temperature: 0` structured output, `json_object` response

ZERO-INFRA MOCK MODE

Every external dependency has a mock fallback by design. No MongoDB? In-memory mock. No Python? Mock node responses. No API keys? Mock LLM responses. `go run ./cmd/server` starts a

format. Graceful degradation when keys are missing.

fully functional server with valid-looking output.

Market Position

FEATURE	TRUTHSEEKERS	WIKIPEDIA	CHATGPT / CLAUDE	PERPLEXITY
Epistemic layer separation	✓ Enforced at code level	✗ Editorial only	✗ None	✗ None
Evidence gap detection	✓ First-class signal	✗ Not modeled	✗ Not modeled	✗ Not modeled
Language framing detection	✓ Dual-channel output	✗ No systematic detection	✗ Reproduces source framing	✗ Reproduces source framing
Multi-dimensional confidence	✓ 6-axis vectors	✗ Binary "true"/needs citation	✗ Scalar only	✗ None
Suppressed evidence modeling	✓ Explicit truth category	✗ Gated by notability	✗ Training data bias	✗ Search rank bias
Full provenance tracing	✓ Claim → source passage	✓ Citation per sentence	✗ Limited to hallucinated refs	✓ Source urls in sidebar
Open source	✓ MIT (permissive)	✓ CC-BY-SA	✗ Proprietary	✗ Proprietary
Local development	✓ Zero infrastructure needed	✓ Needs MediaWiki stack	✗ API access only	✗ API access only

TAM note: The global encyclopedia market is not the reference. The reference markets are knowledge management systems (\$57B by 2027), AI content generation tools (\$18B by 2028), and research intelligence platforms (\$12B by 2026). Truthseekers sits at the intersection — a structured knowledge engine for organizations that need evidence-based decision support, not chat interfaces.

Current Status & Trajectory

Area	Status	Detail
Go backend	Live	Zero-framework API gateway, native agent (25 iters, 14 tools), DAG engine with cycle detection, MongoDB CRUD (23 ops), Python subprocess bridge, SSE streaming, HMAC JWT auth. <code>go run ./cmd/server</code> starts immediately.
Python workers	Complete	9 epistemic nodes, shared LLM client with 4-provider fallback, deterministic structured output (<code>temperature: 0</code>), mock fallback when no keys configured.
Frontend	Live with fixes in progress	SSE streaming fixed (Flush), loading/error states implemented (12 error.tsx, 7 loading.tsx), block rendering fixed, React.memo optimizations, optimistic mutations. Editorial design system implemented.
DAG→HTTP integration	Pending	DAG engine and Python nodes are complete but the HTTP generate path uses a stub. Wiring into <code>handleGenerateArticle</code> is the next engineering milestone.
Infrastructure	Mostly done	Docker Compose, Makefile for up/down/build/logs. Production secret management pending. MongoDB→PostgreSQL mismatch in compose stack.
Testing	Foundational	6 agent tests, 4 DAG engine tests. No external dependencies. Node-level testing of Python workers pending.
Documentation	Comprehensive	System architecture (ARCHITECTURE.html), system design (VERITAS_V1.3_System_Design.md), epistemic layer contract, loading states audit, migration roadmap.

Technology Stack

Backend

- **Language:** Go 1.22+, stdlib net/http — no web frameworks

- **Storage:** MongoDB (`go.mongodb.org/mongo-driver`) + in-memory mock
- **Auth:** HMAC-SHA256 JWT (dev), unsigned in practice
- **LLM:** OpenAI-compatible API (Groq + DigitalOcean Inference)
- **Workers:** Python 3.11+ subprocesses, JSON over STDIN/STDOUT
- **Concurrency:** Goroutines + `sync.WaitGroup` for tool execution, polling loop for DAG

Frontend

- **Framework:** Next.js 15.5.19, React 19
- **Styling:** Tailwind CSS 4, CSS custom properties
- **State:** TanStack Query (server state), React state (UI state)
- **Animation:** Framer Motion 11.x
- **Charts/Maps:** Mermaid.js, Leaflet (2D maps), React Three Fiber (3D)
- **Streaming:** Native fetch + SSE parser

Key decision — no framework: The Go backend uses zero web frameworks. No Gin, no Chi, no Echo. Routing is hand-dispatch on `http.ServeMux`. This is deliberate — the API surface is small and stable, and framework removal reduces attack surface, build time, and dependency risk. The entire orchestrator compiles in <10 seconds.

Team

ENGINEERING

Solo-built by an experienced full-stack engineer with deep Go systems programming, Python ML/LLM pipeline engineering, and React frontend architecture. The system demonstrates proficiency across the entire stack — from TCP-level HTTP flushing bugs to React rendering performance optimization to epistemic system design theory.

DESIGN

Editorial design system built from scratch: antique gold & ink aesthetic, aged-paper backgrounds, Playfair/Lora/Inter typography. Custom block renderer, interactive timeline, 2D/3D map viewers, Mermaid.js diagram integration. Loading states, error boundaries, skeleton screens, optimistic mutations.

EPISTEMIC

Novel three-layer epistemic architecture with typed I/O contracts, enumerated permissions, and hard prohibitions at code level. Claim Graph model with six-axis confidence vectors. Gap detection as first-class epistemic signal. Language framing with dual-channel output.

Roadmap

PHASE	TIMELINE	DELIVERABLES	STATUS
1. Architecture Design	Complete	System design doc, epistemic layer contract, architecture diagram	Done
2. Core Implementation	Complete	Go orchestrator, 9 Python nodes, MongoDB adapter + mock mode, 14 agent tools, DAG engine	Done
3. Frontend Integration	2 weeks	Wire real DAG into HTTP generate path, fix remaining SSE edge cases, complete /maps sub-routes	~70% done
4. Epistemic Tuning	3 weeks	LLM quality runs across complex topics, latency profiling, prompt refinement, confidence calibration	Pending
5. PostgreSQL Migration	2 weeks	Port from MongoDB to PostgreSQL, reconcile docker-compose, retire legacy TS packages	Pending
6. Production Hardening	3 weeks	Real JWT signing, rate limiting, input validation, secret management, deployment automation	Pending

Key Metrics

~15k

~8k

LINES OF GO

LINES OF PYTHON

~25k

LINES OF TYPESCRIPT/JS

19

GO SOURCE FILES

14

AGENT TOOL DEFINITIONS

10

PYTHON WORKER SCRIPTS

23

DATABASE CRUD METHODS

4

LLM PROVIDER GATEWAYS

4

TRUTH CATEGORIES

6

CONFIDENCE AXES

9

DAG NODE OPERATIONS

3

EPISTEMIC LAYERS

Truthseekers Project

veritas/docs/EXECUTIVE_SUMMARY.html

June 24, 2026 · Confidential